

汇编语言

13 参数传递与多模块程序结构

张青

ieqzhang@zzu.edu.cn

实验七 子程序结构

- 1、编写汇编语言程序 exp0701.s。要求：编写一个子程序，它以二进制形式显示EAX中的32位数据，并设计一个主程序验证，其中EAX= 8F98FF00H。
- 2、编写汇编语言程序 exp0702.s。要求：利用十六进制字节显示子程序DISPHB设计一个从低地址到高地址逐个字节显示某个主存区域内容的子程序DISPMEM。入口参数：EAX=主存偏移地址，ECX=字节个数（主存区域的长度）。同时编写一个主程序进行验证，其中要以十六进制字节形式显示的字符串设置为 “This is a test!”。
- 3、编写汇编语言程序 exp0703.s。
 - (a) 编写子程序dprflags，用于显示当前6个状态标志位：CF（进位标志）、ZF（零标志）、SF（符号标志）、OF（溢出标志）、AF（辅助进位标志）、PF（奇偶标志），同时编写一个主程序进行验证。
 - (b) 显示规则：若标志位为1，则输出对应的符号（C、Z、S、O、A、P）；若标志位为0，则输出一个点（.）；
 - (c) 输出格式：连续输出 6 个字符（顺序固定为 CF ZF SF OF AF PF），末尾输出换行符 \n。例如：若 CF=1, ZF=0, SF=1, OF=0, AF=1, PF=0，则输出字符串 "C.S.A.\n"。



实验七 子程序结构

注意：主程序必须先通过以下固定代码段将标志位设置为CF=1, ZF=0, SF=1, OF=0, AF=1, PF=0, 然后调用子程序dprflags显示结果；

设置标志位的固定代码（学生必须使用此段，否则测评可能失败）

```
pushfq
pop  rax
mov  rbx,rax
or   ax, 0x0091    # 设置 CF(bit0)=1, AF(bit4)=1, SF(bit7)=1
and  ax, 0xF7BB    # 清除 ZF(bit6)=0, PF(bit2)=0, OF(bit11)=0
push rax
popfq
```

实验七 子程序结构



4、编写汇编语言程序 exp0704.s。要求：使用Eratosthenes筛法，求1-100 000之间共有多少个素数。

Eratosthenes筛法是希腊数学家Eratosthenes发明的算法，给出了一种找出给定范围内所有素数的快速方法。该算法要求创建一个字节数组，以如下方式将不是素数的位置标记出来：2是一个素数，从位置2开始，把所有2的倍数的位置标记1；2之后的素数3，同样将3的倍数的位置标记1；3之后下一个素数是5，同样将5的倍数的位置标记1；如此重复，直到标记所有非素数的位置。处理结束，数组中没有被标记的位置都对应一个素数。

素数判断程序-1



(1) 采用直接简单的算法：假设输入N，将其逐个除以2 ~ N-1，只要能整除（余数为0）说明不是素数，只有都不能整除才是素数。

(1) 核心程序片段如下：

```
cmp eax,3
```

```
jbe doney
```

;小于等于3的都是素数

```
cdq
```

```
mov esi,eax
```

;EDI.ESI=N>3

```
mov edi,edx
```

```
mov ebx,2
```

;EBX从2开始

素数判断程序-2



again: div ebx

test edx,edx

jz donen

;余数为0, 能整除, 不是素数

inc ebx

cmp ebx,esi ;EBX到N-1结束

mov eax,esi

mov edx,edi

jb again

素数判断程序-3



(2) 采用只对奇数整除的算法：1、2和3是素数，所有大于3的偶数不是素数，从5开始的数字只要除以从3开始的奇数，只有都不能整除才是素数。

- 提示：
- 排除4的情况，从3开始除，每次循环加2

```
mov ebx,3
```

```
.....
```

```
add ebx,2
```

- 子程序设计
- 参数传递方式

教材章节：5.3.3-5.3.5

慕课：9.2-9.3

- 多模块程序结构

教材章节：5.4

慕课：10.1-10.4

- ▶ 堆栈的作用
- ▶ 寄存器传送
- ▶ 共享变量传递
- ▶ 堆栈传递

- ▶ 模块连接
- ▶ 子程序库
- ▶ 源文件包含

学习目标



- 能画出堆栈在子程序调用和返回中的作用
- 会设计子程序，并调用
- 会使用三种参数传递方式
- 能说出模块间共享变量或子程序的伪指令,文件包含的伪指令
- 记忆模块连接的过程和注意事项、子程序库生成和调用的过程及其注意事项
- 会阅读分析子程序结构程序的功能、能说出运行结果
- 会分析多模块程序的功能和执行结果，并能说明开发过程
- 能说出程序开发过程中出现问题的原因并解决
- 养成良好的编程习惯，团队合作，互相帮助的精神



堆栈传递参数

- 主程序将入口参数压入堆栈，子程序从堆栈中取出参数
- 出口参数通常不使用堆栈传递
- 高级语言进行函数调用时提供的参数，实质也利用堆栈传递
- 采用堆栈传递参数是程式化的，它是编译程序处理参数传递、以及汇编语言与高级语言混合编程时的常规方法

〔例5-25〕 计算有符号数平均值程序



```
.data                                #定义数据段
array: .long 675, 354, -34, 198, 267, 0, 9, 2371, -67, 4257
.equ len, (.-array)/4
.text                                #定义代码段
.globl main
main:                                #程序执行起始位置
    push len                         #压入数据个数
    lea rax, array[rip]
    push rax                         #压数组的偏移地址
    call mean                        #调用求平均值子程序, 出口参数: EAX = 平均值 (整数部分)
    add rsp, 16                      #平衡堆栈 (压入了16个字节数据)
    call dispsid                     #显示
    mov eax, 0
    ret
```

〔例5-25〕 计算有符号数平均值程序



mean:

```
push rbp
mov rbp, rsp
push rbx
push rcx
push rdx
```

```
mov rbx, [rbp+16]
```

```
mov rcx, [rbp+24]
```

```
xor eax, eax
```

```
xor rdx, rdx
```

#计算32位有符号数平均值子程序

#入口参数：顺序压入数据个数和数组偏移地址

#出口参数：EAX = 平均值

#保护寄存器

#EBX = 堆栈中取出的偏移地址

#ECX = 堆栈中取出的数据个数

#EAX保存和值

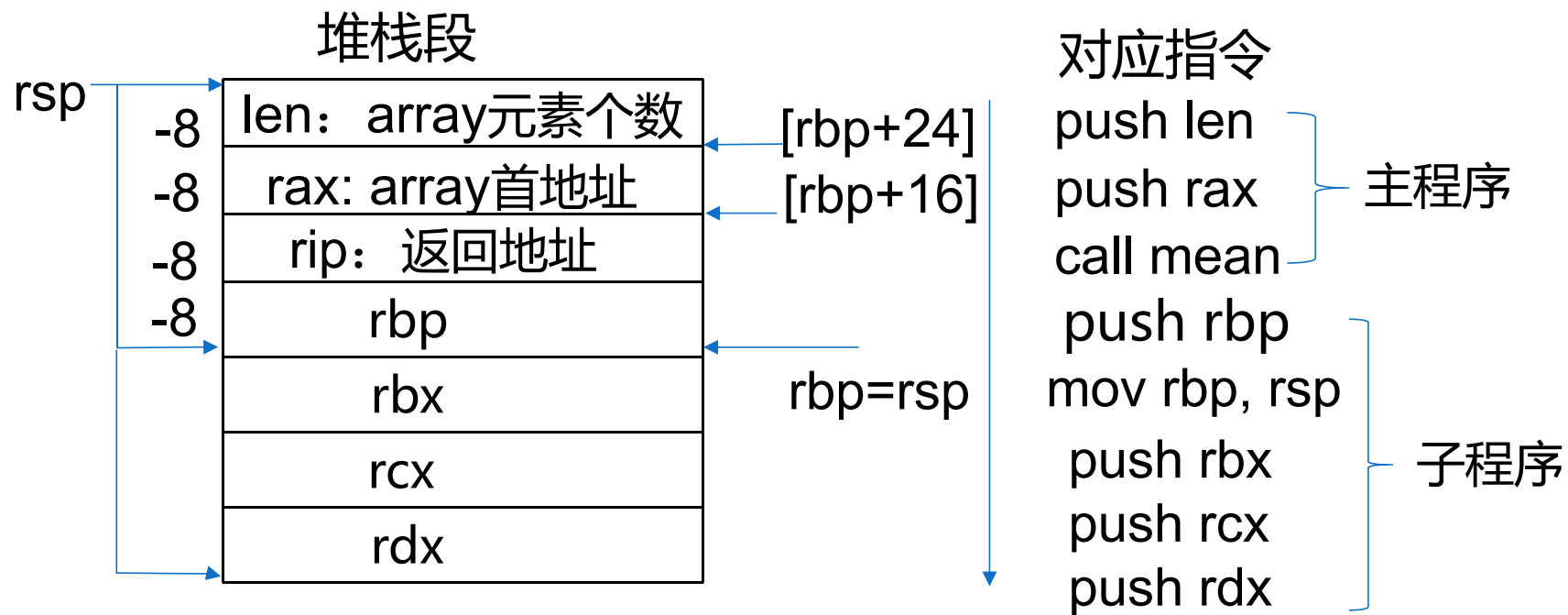
#EDX = 指向数组元素

〔例5-25〕 计算有符号数平均值程序



```
mean1:add eax,[rbx+rdx*4]#求和
      add edx,1          #指向下一个数据
      cmp edx,ecx        #比较个数
      jb mean1          #循环
      cdq               #将累加和EAX符号扩展到EDX
      idiv ecx           #有符号数除法, EAX = 平均值 (余数在EDX中)
      pop rdx           #恢复寄存器
      pop rcx
      pop rbx
      pop rbp
      ret
```

〔例5-25〕堆栈传递参数



〔例5-25〕平衡堆栈



等价于子程序中 ret 16

对应指令

主程序 {

add rsp,16

子程序 {

ret

pop rdx

pop rcx

pop rbx

pop rbp

	字节数	堆栈段
+16	8	len: array元素个数
	8	rax: array首地址
+8	8	rip: 返回地址
+32	8	rbp
	8	rbx
	8	rcx
	8	rdx
rsp		

改进的子程序：防止溢出



mean_v2:

push rbp

mov rbp, rsp

push rbx

push rcx

push rdx

push rsi

push rdi

mov rbx, [rbp+16]

mov rcx, [rbp+24]

xor rsi, rsi

mov rdi, rsi

mean1_v2:

mov eax, [rbx]

cdq

#改进计算32位有符号数平均值子程序

#入口参数：顺序压入数据个数和数组偏移地址

#出口参数：EAX = 平均值

#保护寄存器

#EBX = 堆栈中取出的偏移地址

#ECX = 堆栈中取出的数据个数

#ESI = 求和的低32位值

#EDI = 求和的高32位值

#EAX = 取出一个数据

#EAX符号扩展到EDX

改进的子程序：防止溢出



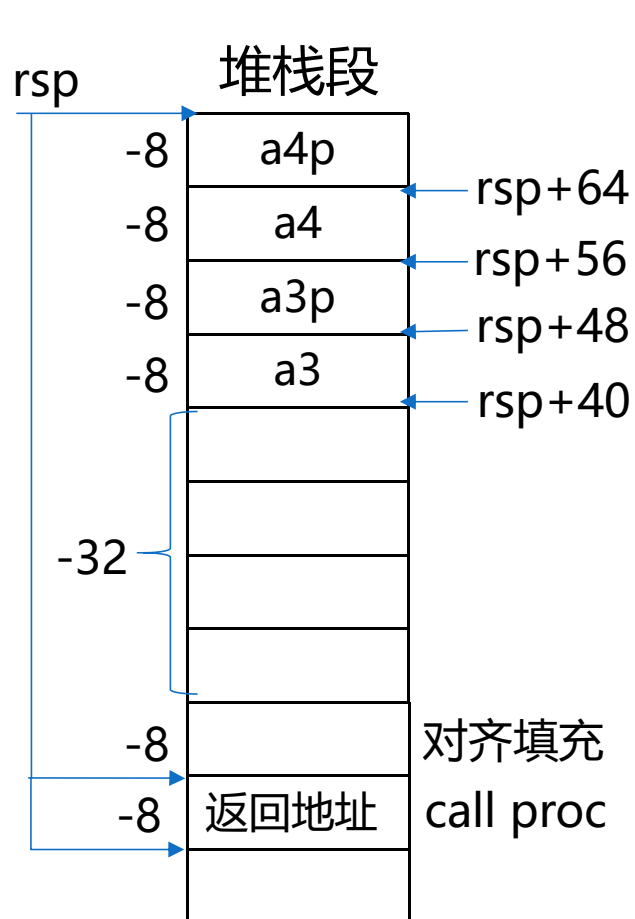
```
add esi,eax      #求和低32位
adc edi,edx      #求和高32位
add rbx,4        #指向下一个数据
dec ecx          #数据个数减少一个
jnz mean1_v2     #循环（这两条指令等同于LOOP指令）
mov eax,esi      #累加和在EDX.EAX
mov edx,edi
idiv dword ptr [rbp+24] #有符号数除法，EAX = 平均值（余数在EDX中）
pop rdi          #恢复寄存器
pop rsi
pop rdx
pop rcx
pop rbx
pop rbp
ret
```

ABI规定的参数传递



- Windows
 - 32位Windows ABI规定函数调用时提供的参数要通过堆栈传递
 - 64位Windows ABI则规定前四个参数用RCX,RDX,R8,R9传递, 其他参数通过堆栈传递
 - 调用者还需要在堆栈预留32个字节的影子空间 (Shadow Space)
 - 保持栈指针RSP16字节对齐。
 - 返回值通常用 RAX (整数) 或 XMM0 (浮点) 传递。
- Linux
 - 前6个整数参数用寄存器 RDI, RSI, RDX, RCX, R8, R9传递
 - 其他参数用堆栈传递
 - 返回值通常用 RAX (整数) 或 XMM0 (浮点) 传递。
 - 保持栈指针RSP16字节对齐。

Windows ABI规定的参数传递示例



C语言函数

```
void proc(long a1, long *a1p,
          int a2, int *a2p,
          short a3, short *a3p,
          char a4, char *a4p)
{
    *a1p += a1;
    *a2p += a2;
    *a3p += a3;
    *a4p += a4;
}
```

a1 : rcx
a1p : rdx
a2 : r8
a2p : r9

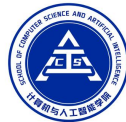
编译

汇编子程序

```
proc:
mov     r10d, ecx
mov     rcx, rdx
mov     rdx, QWORD PTR 48[rsp]
mov     rax, QWORD PTR 64[rsp]
add     DWORD PTR [rcx], r10d
add     DWORD PTR [r9], r8d
mov     ecx, DWORD PTR 40[rsp]
add     WORD PTR [rdx], cx
mov     edx, DWORD PTR 56[rsp]
add     BYTE PTR [rax], dl
ret
```

调用者创建栈帧

sub rsp, 72 (不考虑局部变量)



合理选择参数传递方式

- 寄存器参数传递
 - 简捷，适合少量参数传送
- 共享变量参数传递
 - 通用性不好
 - 全局变量
- 堆栈传递
 - 普遍使用，复杂，注意堆栈的使用

练习：保存寄存器与子程序返回



#计算32位有符号数平均值子程序,
#入口参数: ebx=数组首地址, ecx=元素个数
#出口参数: eax=平均值
mean:

```
push rax
push rbx                #保护寄存器
push rcx
push rdx
mov rdx,0
#...求[ebx]中存放的ecx个数据的平均值
#EAX = 平均值
(   ?   )
(   ?   )                #恢复寄存器
                        #返回
```

A:
pop rdx
pop rcx
pop rbx
ret

B:
pop rdx
pop rcx
pop rbx
pop rax
ret

C:
pop rdx
pop rcx
pop rbx
ret 8

D:
pop rbx
pop rcx
pop rdx
ret 8



子程序的局部变量

- 使用局部变量可以提高程序的模块化程度。
- 局部变量仅在子程序内部可以使用，不能被子程序外的其他代码使用。
- 子程序中的局部变量保存在寄存器或堆栈中。
- 保存参数和局部变量的堆栈区域被称为堆栈帧（Stack Frame）
 - 在函数调用时建立、返回后消失。
- 局部变量的作用域是子程序。
- 局部变量的实现原理：
 - 在进入子程序的时候，通过修改堆栈指针RSP来预留出局部变量需要的空间；
 - 返回主程序之前，通过恢复RSP释放这些空间，在堆栈中不再为局部变量保存空间。

【例5-26】 swap_add函数汇编

```
long swap_add(long *xp, long *yp)
{
    long x = *xp;
    long y = *yp;
    *xp = y;
    *yp = x;
    return x + y;
}
```

编译

```
swap_add:
    mov     eax, DWORD PTR [rcx]
    mov     r8d, DWORD PTR [rdx]
    mov     DWORD PTR [rcx], r8d
    mov     DWORD PTR [rdx], eax
    add     eax, r8d
    ret
```

xp: rcx
yp: rdx
x: eax
y: r8d
x+y: eax

} 局部变量在寄存器中

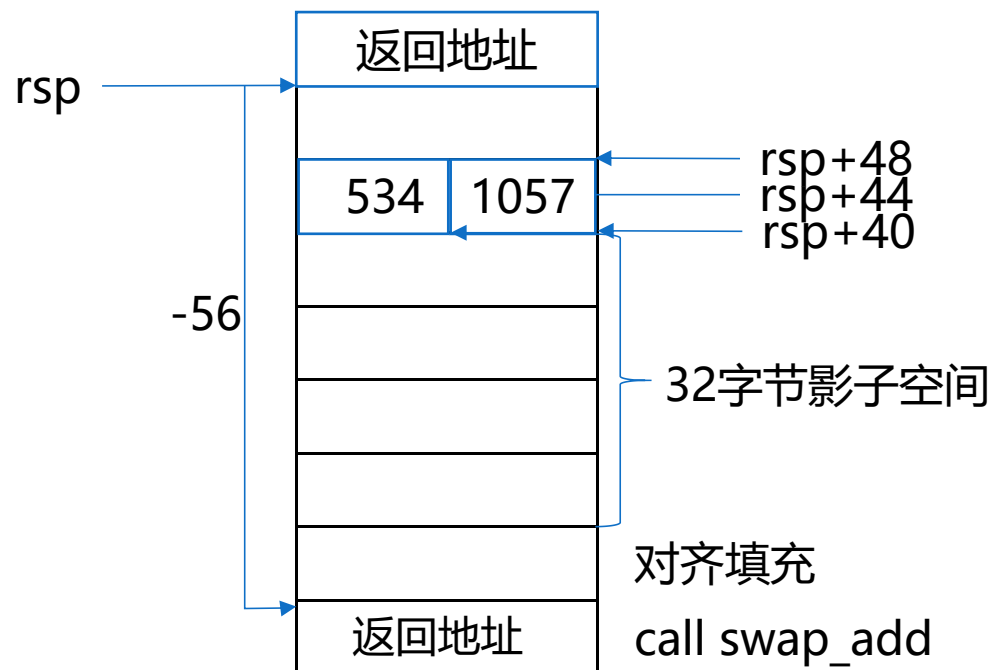
函数执行后eax和r8d的值变化

〔例5-26〕 caller函数汇编

```
long caller()
{ long arg1 = 534;
  long arg2 = 1057;
  long sum = swap_add(&arg1, &arg2);
  long diff = arg1 - arg2;
  return sum * diff;
}
```

编译

```
caller:
    sub     rsp, 56
    mov     DWORD PTR 44[rsp], 534
    mov     DWORD PTR 40[rsp], 1057
    lea     rdx, 40[rsp]
    lea     rcx, 44[rsp]
    call    swap_add
    mov     edx, DWORD PTR 44[rsp]
    sub     edx, DWORD PTR 40[rsp]
    imul    eax, edx
    add     rsp, 56
    ret
```



arg1: rsp+44 } 局部变量在堆栈
 arg2: rsp+40 }
 sum: eax } 局部变量在寄存器
 diff: edx }
 rdx: &arg2 } 两个实参
 rcx: &arg1 }



多模块程序结构

多模块程序结构



- 程序分段、子程序等实现了程序模块化
- 开发大型应用程序时常使用
 - 多个源程序文件
 - 目标代码模块等
- 形成多模块程序结构

GAS支持的多模块方法

- ✓ 模块连接
- ✓ 子程序库
- ✓ 源文件包含

模块连接

- 子程序单独编写，汇编形成目标模块文件
- 连接时输入子程序模块文件名
- 用共用伪指令 **.global** 和外部伪指令 **.extern** 声明
 - .global 标识符** # 定义标识符的模块使用
 - .globl 标识符** # 定义标识符的模块使用
 - .extern 标识符** # 调用标识符的模块使用
- 在 GAS 中，**.extern** 通常可以省略，因为汇编器会自动处理未定义的符号。
- 子程序必须在代码段中
- 处理好子程序与主程序之间的参数传问题

〔例5-27〕 全局标识符声明



```
//eg0527.c
void print(char *);
int main(){
char s[] = "Hello";
print(s);
return 0;
}
```

```
// eg0527s.c
#include
<stdio.h>
void print(char* s){
    printf(s);
}
```

```
gcc -S -Og -masm=intel -o ./progs/eg0527.s ./progs/eg0527.c
```

```
gcc -S -Og -masm=intel -o ./progs/eg0527s.s ./progs/eg0527s.c
```

〔例5-27〕 eg0527.s



```
.text
.....
.globl main
.....
main:
    sub    rsp, 56
    .....
    mov    DWORD PTR 42[rsp], 1819043144 # 写入 "Hell"
    mov    WORD PTR 46[rsp], 111         # 写入 "o\0"
    lea     rcx, 42[rsp]                 # 入口参数rcx
    call   print
    mov     eax, 0
    add     rsp, 56
    ret
    .....
```

〔例5-27〕 eg0527s.s



```
        .text
.....
        .globl  print
.....
print:
        sub     rsp, 40
        call    printf    #入口参数rcx
        nop
        add     rsp, 40
        ret
.....
```

```
gcc -o eg0527 ./progs/eg0527.S ./progs/eg0527s.S
```

执行可执行文件eg0527:

```
>eg0527  
Hello
```

```
>gcc -c -o ./progs/eg0527.o ./progs/eg0527.S
```

```
>gcc -c -o ./progs/eg0527s.o ./progs/eg0527s.S
```

```
>gcc -o eg0527 ./progs/eg0527.o ./progs/eg0527s.o
```

〔例5-28〕 标志位显示程序



```
# eg0528.s
.intel_syntax noprefix
.text
.global main
main:
    sub    rsp, 32
    mov    al, 0x80
    sub    al, 1
    call   dprf
    add    rsp, 32
    mov    eax, 0
    ret
```

```
RFLAGS = 00000A12 O
RFLAGS = 00000293 C S
```


〔例5-28〕 eg0528s.s



.text

.global dprf

#dprf声明为全局函数，可被其他模块调用

dprf:

pushf

pop rax

#rax=rflags

sub rsp, 72

#建立栈帧

mov 32[rsp],rax #保存标志寄存器

mov dword ptr 40[rsp], 0x414C4652 #字符串"RFLAGS ="

mov dword ptr 44[rsp], 0x3D205347

lea rdi, 48[rsp] #eax对应的十六进制字符串首地址

call hdtoasc # eax转换为十六进制形式字符串

〔例5-28〕 eg0528s.s-2



```
mov eax, 32[rsp]          #恢复标志寄存器
lea rsi, 56[rsp]
test al, 1                #判断CF=1?
jz L1
mov word ptr [rsi],0x4320 #CF=1
add rsi, 2
L1: test al, 4            #判断PF=1?
jz L2
mov word ptr [rsi],0x5020 #PF=1
add rsi, 2
L2: test al, 0x40         #判断ZF=1?
jz L3
mov word ptr [rsi],0x5a20 #ZF=1
```

〔例5-28〕 eg0528s.s-3



add rsi, 2	
L3: test al, 0x80	#判断SF=1?
jz L4	
mov word ptr [rsi], 0x5320	#SF=1
add rsi, 2	
L4: test ax, 0x800	#判断OF=1?
jz L5	
mov word ptr [rsi], 0x4f20	#OF=1
add rsi, 2	
L5: mov byte ptr [rsi], 0	#字符串结束
lea rcx, 40[rsp]	#调用printf的入口参数
call printf	
add rsp, 72	#释放栈帧
ret	

〔例5-28〕 eg0528s.s-4



```
hdtoasc:                # eax转换为十六进制形式字符串
    xor rbx,rbx          #使用rbx相对寻址访问REGD字符串
    mov ecx,8            # 8位十六进制数
again:rol eax,4           #高4位循环移位进入低4位，作为子程序的入口参数
    push rax             #子程序利用AL返回结果，所以需要保存EAX中的数据
    call htoasc          #调用子程序
    mov [rdi+rbx],al     #保存转换后的ASCII码
    pop rax              #恢复保存的数据
    inc rbx
    loop again
ret
```

〔例5-28〕 eg0528s.s-5



```
htoasc:          #将AL低4位表达的一位十六进制数转换为ASCII码
    and al,0xf    #只取AL的低4位
    or al,0x30    #AL高4位变成3，实现加30H
    cmp al,0x39   #是0~9，还是A~F
    jbe htoend
    add al,7      #是A~F，其ASCII码再加上7
htoend: ret       #子程序返回
```

模块连接的操作过程

- 子程序单独编写一个源程序文件
- 子程序源文件汇编形成目标模块OBJ文件
- 连接时输入子程序目标模块文件名

操作过程

```
gcc -c -o eg0528.o ./progs/eg0528.s
```

```
gcc -c -o eg0528s.o ./progs/eg0528s.s
```

```
gcc -o eg0528 eg0528.o eg0528s.o
```

或:

```
>gcc -o eg0528 ./progs/eg0528.S ./progs/eg0528s.S
```

```
>eg0528
```

```
RFLAGS =00000A12 O
```